Comprehensive Fuzzy Transformer Study

A plain-language comparison of fuzzy curves, Mamdani rules, attention types, model sizes, input lengths, and three Transformer designs

What we tested	Best tested Transformer	Final-test RMSE	Important result
72 declared cases 64 unique models	10 past values, 1 layer, 7 rules parabolic -> triangular	**1.4436 +/- 0.0450**	Simple persistence is better (1.3727)

Executive verdict

Among the Transformer models we tested, the winner was a **standard-attention custom Transformer with one encoder layer, 10 past values, and seven Mamdani rules**. It uses parabolic input curves and triangular output curves. The fuzzy step comes after the feed-forward step. On the stronger validation test, its RMSE was **1.6512 +/- 0.0775**. Lower RMSE is better. It came first, but only by a small amount.

On the final 374-example test, its average RMSE across three runs was **1.4436**. It was 32.60% better than our reproduced BCM result. However, it was 5.17% worse than simply copying the latest value, 1.21% worse than the saved API Transformer run, and 2.65% worse than the saved FANTF run. The API and FANTF numbers each come from only one run, so they are useful background information, not a perfectly fair statistical comparison.

The short answer: the standard-attention Mamdani Transformer was the best Transformer we tested. Persistence - copying the latest value - was still the best forecast overall. Fuzzy layers can work, but adding more fuzzy machinery does not automatically improve accuracy.

1. Research question and evaluation target

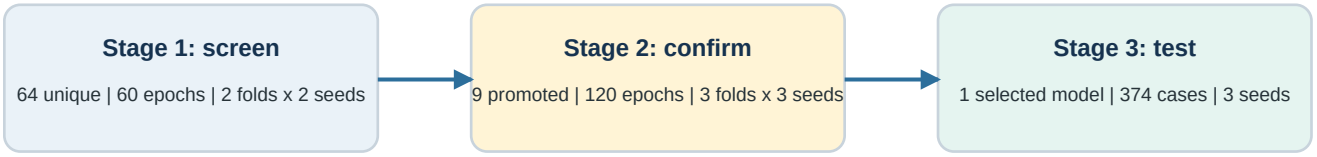
The original ieRSPOP workflow uses BCM to learn an association. We replace that BCM learning step with a Transformer that we can inspect and change ourselves. The task is simple: give the model a short list of past volatility values and ask it to predict the next value.

Questions answered

#	Research question
1	Which pair of fuzzy input and output curves gives the lowest error?
2	How many Mamdani rules, Transformer layers, and past values should we use?
3	Should the fuzzy step come after attention, after feed-forward processing, or in both places?
4	Does FANTF random fuzziness or distance-based fuzzy attention beat normal attention?
5	How do Transformer, PatchTST-style, and iTransformer-style models compare on this one-variable dataset?

Dataset and scored protocol

Item	Setting
Examples available	860 examples for predicting the next value
Final untouched test	374 examples; starts at recorded MATLAB window 487
Order	Predict first, show the correct answer, then let the model learn
Scaling	Uses only information that was already available at prediction time
Scores	RMSE and MAE: lower is better. Correlation and R-squared: higher is better
Simple comparison	Persistence: copy the latest observed value as the next prediction

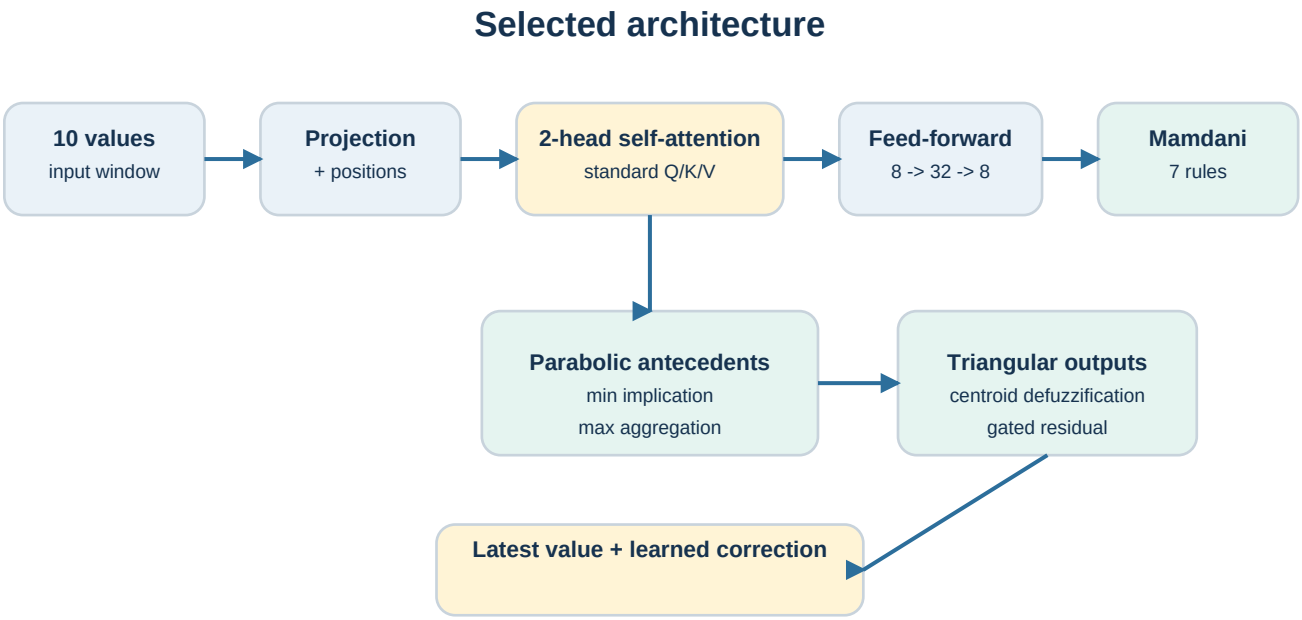


The held-out test is consulted only after validation-based selection.

We choose the model before looking at the final test. First we do a quick wide search, then a stronger recheck of the finalists, and only then do we test the single winner. This keeps the final result fair.

2. Models we built

Selected model



We wrote the Transformer parts ourselves using basic PyTorch building blocks. This includes the query, key, and value (Q/K/V) calculations, splitting into attention heads, attention scoring, softmax, skip connections, normalization, and the feed-forward network. We did not call PyTorch's ready-made Transformer or multi-head-attention modules.

Version	What it does	Why we tested it
Custom Transformer	Treats each time point as one token and uses normal attention	Main reference model
FANTF-style	Adds controlled random fuzzy noise to attention scores	Checks whether random fuzziness helps
Deterministic fuzzy	Uses query-key distance to make a fuzzy similarity score	Checks fuzziness without randomness
PatchTST-style	Groups nearby values into overlapping patches	Checks whether small chunks work better
iTransformer-style	Treats each variable as a token	Checks inverted tokens; here we have only one variable

Our PatchTST-style and iTransformer-style versions reuse our small custom encoder. They are not exact copies of the full official research code. iTransformer normally compares several variables, but this dataset has only one. That gives it only one variable token, so it cannot use its main advantage.

3. Mamdani fuzzy layer and curve shapes

The Mamdani layer turns each hidden number into fuzzy ideas such as low, medium, and high. It applies rules to those ideas, combines the rule outputs, and turns them back into one normal number using the center of the output shape. A learned gate controls how much this fuzzy answer changes the original hidden value. The centers and widths can improve during training.

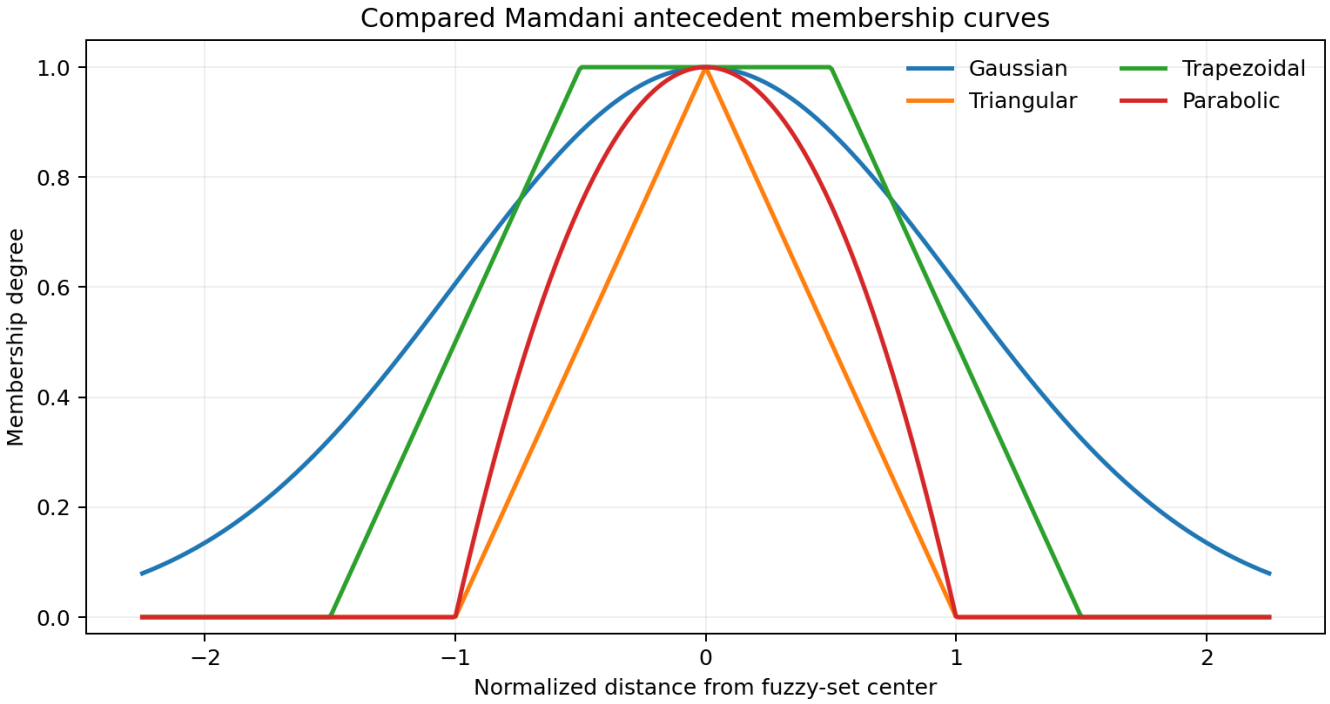


Figure 1. The four fuzzy curve shapes tested for inputs and outputs.

Curve	Formula for distance d	Easy meaning
Gaussian	exp(-0.5 d^2)	Smooth bell curve; fades slowly
Triangular	max(1 - |d|, 0)	Straight sides and a sharp top
Trapezoidal	clip(1.5 - |d|, 0, 1)	Flat top with straight sides
Parabolic	max(1 - d^2, 0)	Rounded curve that reaches zero

We tested every possible pair of the four input curves and four output curves: 16 pairs in total. The model learns the input-curve positions and widths. The output shapes stay fixed, which makes it easier to understand what each fuzzy rule is doing.

4. Experiment matrix

Experiment group	Choices compared	Cases
Main combinations	Mamdani or FANTF+Mamdani; rules 3/5/7; layers 1/2; windows 5/10/20	36
Curve pairs	4 input curves combined with 4 output curves	16
Fuzzy placement	After attention, after feed-forward, both	3
Encoder depth	1, 2, 3 layers	3
Sequence length	5, 10, 20, 30 observations	4
FANTF delta	0.1, 0.25, 0.5, 1.0	4
Attention type	Normal, FANTF random fuzzy, distance-based fuzzy	3
Model family	Transformer, PatchTST-style, iTransformer-style	3
Total	72 declared; duplicate base settings cached	64 unique

Training budgets

Stage	Training rounds	Time splits	Random seeds	Results per model
Quick search	60	2	42, 52	4 results
Stronger recheck	120	3	42, 52, 62	9 results
Final test	120	374 test cases	42, 52, 62	3 full test runs

We moved the best model from each experiment group, plus the five best unique models overall, into the stronger recheck. Models were ranked by average RMSE, where a smaller number is better.

5. Quick-search results

Experiment group	Quick-search winner	RMSE	SD
Main combinations	Mamdani, 3 rules, 1 layer(s), window 10	1.5904	0.0476
Past values used	window 10	1.5904	0.0476
Curve pairs	parabolic -> gaussian	1.6099	0.0470
Attention type	FANTF stochastic	1.6126	0.0539
FANTF noise	delta 0.1	1.6141	0.0530
Number of layers	1 encoder layer(s)	1.6225	0.0542
Fuzzy position	after_feedforward	1.6225	0.0542
Model family	itransformer	1.6507	0.0362

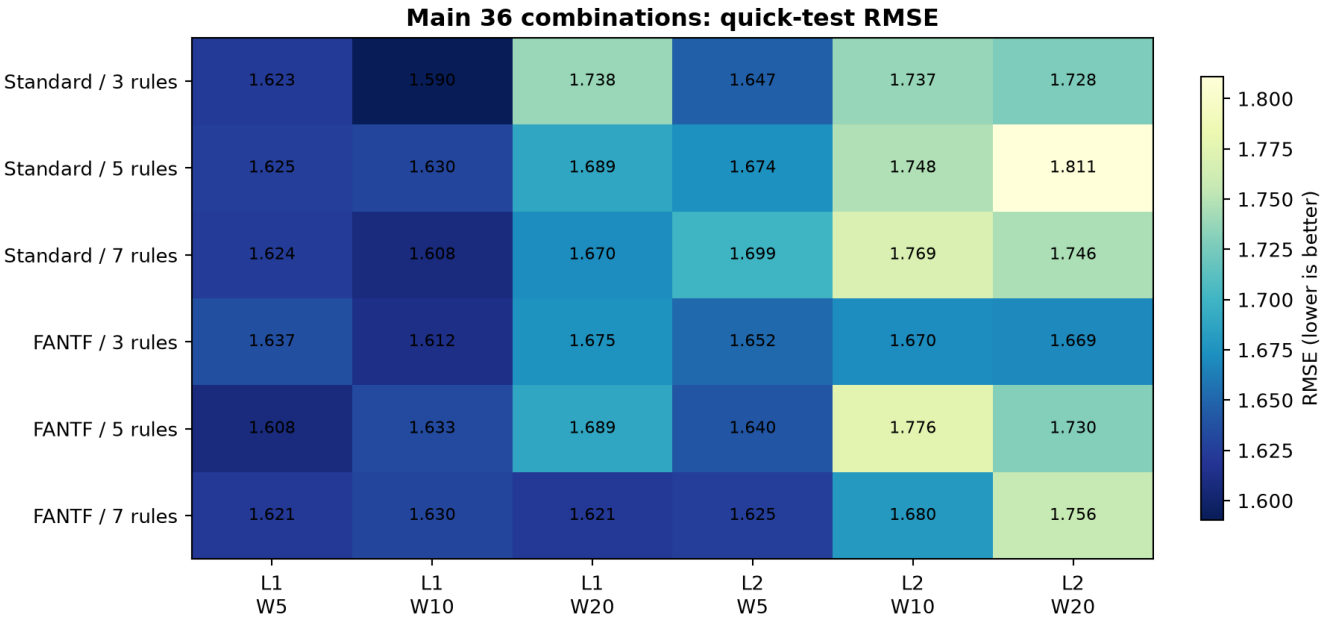


Figure 2. The quick search prefers one layer and 10 past values. Adding more rules or a second layer does not always improve the score. These numbers help us choose finalists; they are not final test results.

6. Comparing the fuzzy curve pairs

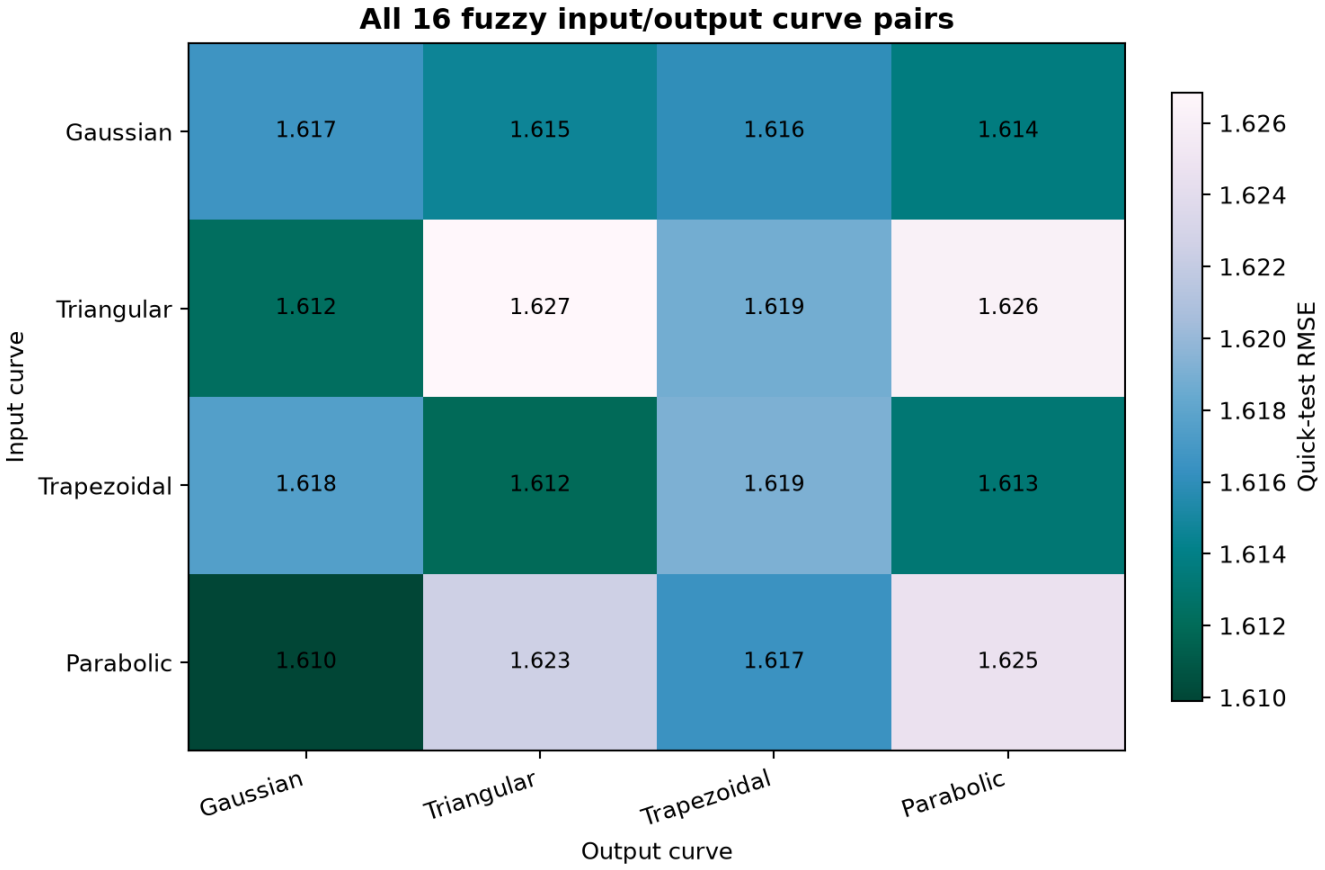


Figure 3. RMSE for all 16 fuzzy input/output curve pairs. In this quick test, parabolic inputs with Gaussian outputs have the lowest error. Parabolic inputs with triangular outputs are still close.

Rank	Input curve	Output curve	RMSE	SD
1	parabolic	gaussian	1.6099	0.0470
2	trapezoidal	triangular	1.6120	0.0427
3	triangular	gaussian	1.6123	0.0456
4	trapezoidal	parabolic	1.6131	0.0449
5	gaussian	parabolic	1.6137	0.0451
6	gaussian	triangular	1.6146	0.0460

Important: the quick winner was not the final winner. Parabolic -> Gaussian won the curve screen but finished sixth in the stronger recheck. The final winner used parabolic -> triangular, seven rules, and 10 past values. Rechecking the finalists stopped us from trusting one lucky quick result.

7. Comparing fuzzy position, attention, layers, history, and model family

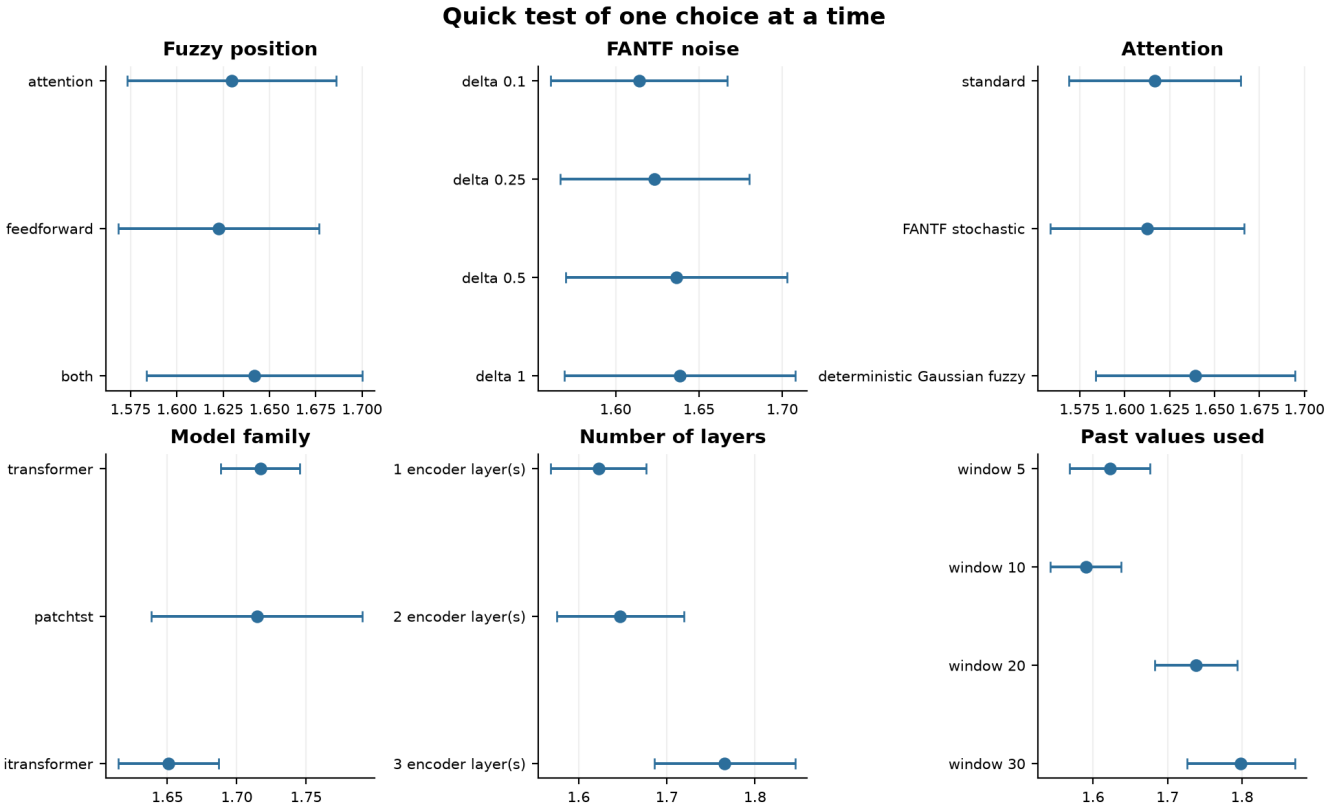


Figure 4. A quick test of one choice at a time. The lines show how much results changed across two time splits and two random starts.

Choice	What the quick test showed
Fuzzy position	Putting Mamdani after feed-forward worked best. Using it twice added complexity without a clear gain.
FANTF noise	The smallest noise, 0.1, worked best. Stronger noise did not help.
Attention	FANTF won the quick attention test, but FANTF-only finished eighth in the stronger recheck.
Layers	One Transformer layer worked better than deeper models on this small dataset.
Past values	Using 10 past values won both the quick search and final model choice.
Model family	iTransformer-style won its quick test but finished last in the stronger recheck.

Do not use this result to judge iTransformer on normal multi-variable data. We have only one variable, so iTransformer gets only one variable token and cannot compare variables as it normally would.

8. Stronger recheck of the finalists

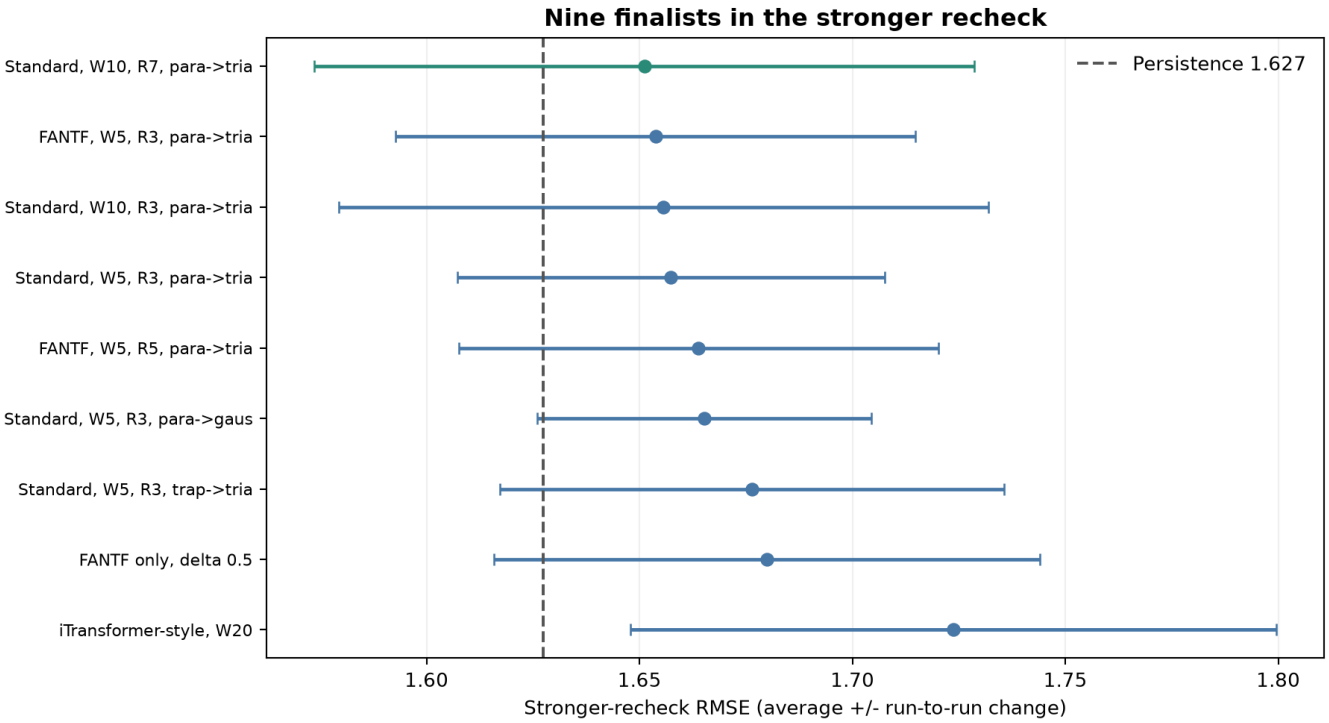


Figure 5. We retrained nine finalists for longer, using three time-based data splits and three random starting points. The dashed line shows persistence on exactly the same data. W means past values, R means rules, para means parabolic, tria means triangular, and gaus means Gaussian.

Rank	Configuration	Params	RMSE	MAE	r
1	standard W10 R7 para->tria	1113	1.6512	1.2896	0.7937
2	fantf W5 R3 para->tria	1010	1.6538	1.2647	0.7888
3	standard W10 R3 para->tria	1049	1.6556	1.2796	0.7920
4	standard W5 R3 para->tria	1009	1.6574	1.2656	0.7893
5	fantf W5 R5 para->tria	1042	1.6638	1.2664	0.7878
6	standard W5 R3 para->gaus	1009	1.6652	1.2767	0.7880
7	standard W5 R3 trap->tria	1009	1.6764	1.2766	0.7852
8	FANTF-only W5	954	1.6799	1.2890	0.7808
9	iTransformer-style W20	1073	1.7237	1.3359	0.7758

The winner beats second place by only 0.16% RMSE. The run-to-run ranges overlap, so we should say it had the best average score, not that it is definitely superior. It is also 1.46% worse than persistence on these validation splits.

9. Final untouched test results

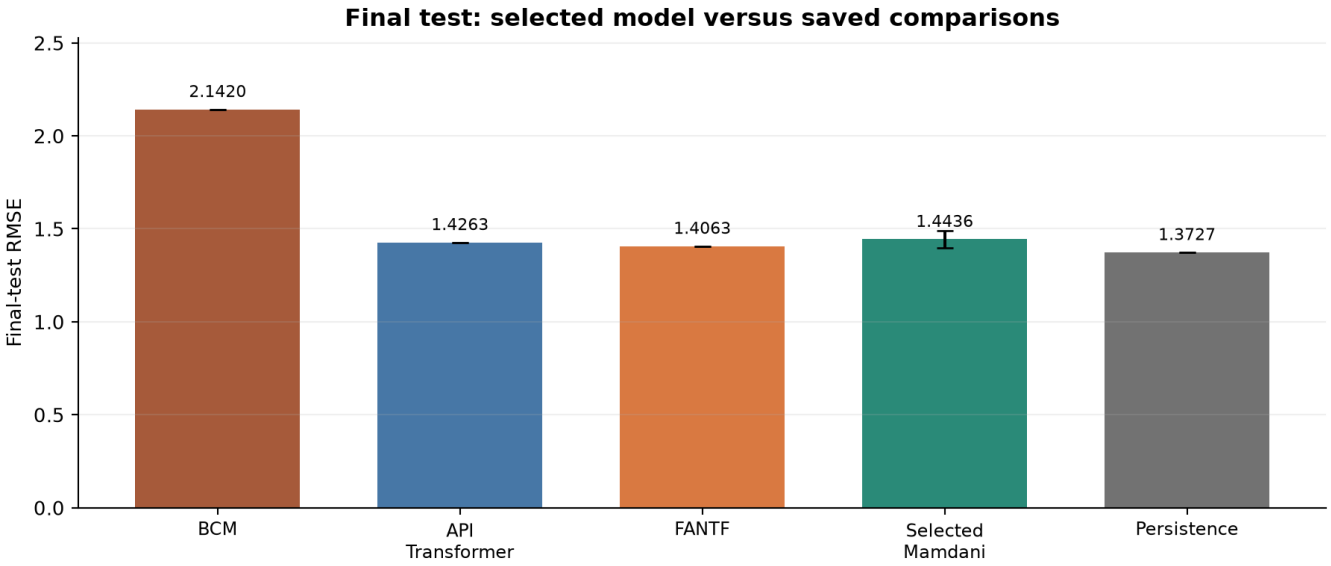


Figure 6. BCM, API Transformer, and FANTF are older saved results from one run each. The selected Mamdani result is the average of three runs; its error bar shows how much those runs changed.

Metric	Selected mean	Seed SD	Persistence	Interpretation
RMSE	1.4436	0.0450	1.3727	Persistence lower
MAE	1.0811	0.0376	0.9158	Persistence lower
Pearson r	0.9431	0.0032	0.9476	Persistence higher
Predictive R2	0.8842	0.0073	0.8954	Persistence higher

Seed	RMSE	MAE	Pearson r	Predictive R2
42	1.5056	1.1214	0.9386	0.8742
52	1.3999	1.0310	0.9459	0.8912
62	1.4255	1.0909	0.9449	0.8872

The selected model has 1113 numbers that training can change. Its RMSE is 32.60% lower than our reproduced BCM result. It does not beat persistence or the two older one-run Transformer results. A separate published/reference ieRSPOP++ RMSE of 1.8045 is also stored in the project, but it is not the same as our reproduced BCM result of 2.1420.

10. Forecast behavior

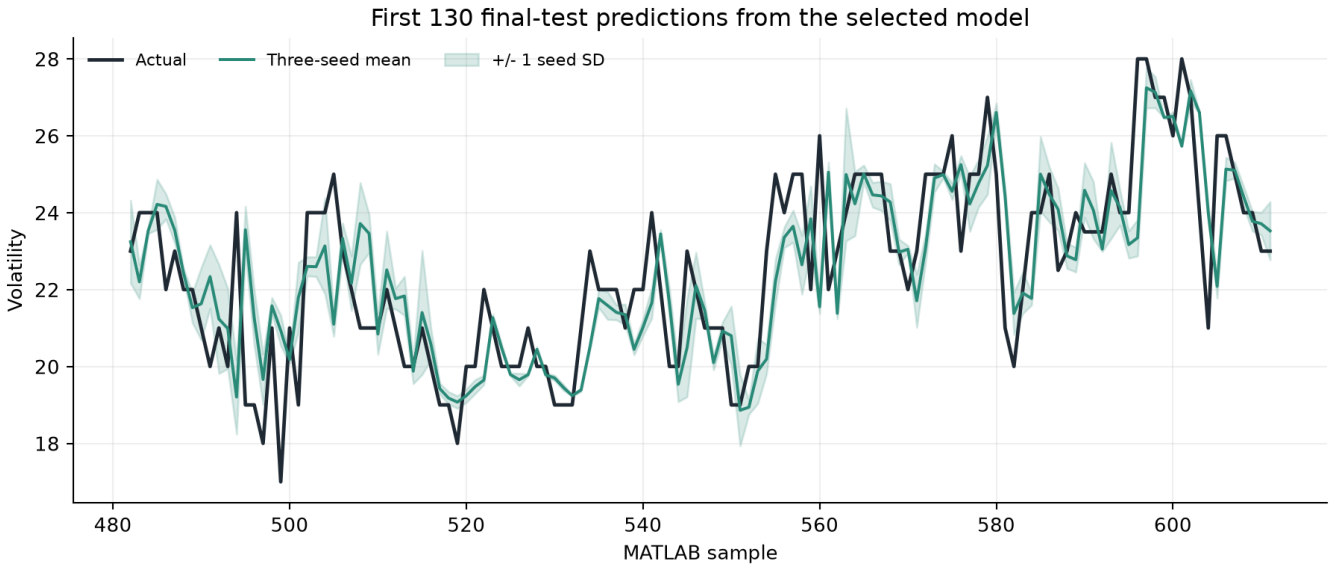


Figure 7. The selected model follows the main movement of volatility, and the three runs are usually close together. Sudden changes are harder to predict. That is one reason why simply copying the latest value is difficult to beat.

Why persistence is hard to beat

Nearby volatility values are usually similar, and we predict only one step ahead. Persistence takes advantage of this by copying the latest value and has nothing to train. Our Transformer also starts from that copied value and learns a correction. With only 860 examples, the correction can sometimes make a good simple prediction worse.

What the Transformer still contributes

The experiment still gives us a trainable and understandable replacement for the BCM learning step. It also shows which fuzzy choices helped and gives us code we can reuse with more variables or more data. Finding that persistence is better is an honest and useful result, not a failed experiment.

11. Limitations and what they mean

Limitation	Why it matters	What to do next
Small one-variable dataset	Deep and multi-variable models cannot show their full strengths.	Add useful variables or test on a multi-variable dataset.
Many models tested	A model can look good by luck, even on validation data.	Use another untouched dataset or a validation-inside-validation design.
Only two seeds in quick search	Early rankings can change.	We reduced this problem by rechecking finalists with three seeds and three splits.
Older results are not fully matched	API and FANTF use one run, while our selected model uses three.	Rerun every learned baseline with the same settings and seeds.
Style adaptations	Our PatchTST/iTransformer versions are not exact official-code copies.	Use the official repos before making publication-level claims.
One-variable iTransformer	One variable creates only one attention token.	Retest it with several real input variables.
No formal significance test	Overlapping ranges make a strong superiority claim unsafe.	Compare paired errors and run a suitable paired statistical test.

Claims that are supported

#	Supported statement
1	We implemented and tested every fuzzy and Transformer group listed in the experiment plan.
2	Among tested Transformers, the 10-value, seven-rule parabolic/triangular Mamdani model has the lowest confirmed average RMSE.
3	Extra layers, stronger FANTF noise, and extra fuzzy positions did not reliably help this dataset.
4	Across every method, persistence has the lowest final-test RMSE in the current evidence.

12. Conclusion: which model gave the best result?

We did more than compare four curve shapes. We tested 64 unique models, rechecked nine finalists, and used the final untouched data only for the single selected model. **The best tested Transformer was the standard-attention Mamdani Transformer**: one encoder layer, 10 past values, seven rules, parabolic input curves, triangular output curves, and the fuzzy step after feed-forward processing. Its final-test RMSE was **1.4436**. **The best result overall was persistence**, with RMSE **1.3727**, because lower RMSE is better.

Suggested 45-second explanation

We built the Transformer parts ourselves using basic PyTorch tools and added a Mamdani fuzzy system inside the encoder. We tested 72 planned cases covering curves, rule counts, fuzzy position, attention, layers, history length, and three Transformer styles. We chose models using earlier time-based data and used the final test only after choosing the winner. The best Transformer was the normal-attention Mamdani model with one layer, ten past values, seven rules, parabolic input curves, and triangular output curves. It was much better than our reproduced BCM result. However, persistence - simply copying the latest value - was still best overall. Therefore, our fuzzy Transformer is a useful learned BCM replacement, but it is not yet the most accurate forecast for this small dataset.

Recommended next experiment

Keep the winning design fixed. Then rerun persistence, the API Transformer, FANTF-only, and the winning Mamdani model with at least five identical random seeds and the same time splits. Compare the errors from matching runs. After that, test on a volatility dataset with several input variables so PatchTST and iTransformer have a fairer chance to use their main strengths.

Appendix A. Complete screening summary

Family	Configuration	Params	RMSE	SD
attention type	FANTF stochastic	954	1.6126	0.0539
attention type	standard	953	1.6168	0.0477
attention type	deterministic Gaussian fuzzy	955	1.6393	0.0554
backbone	itransformer	1073	1.6507	0.0362
backbone	patchtst	1009	1.7145	0.0762
backbone	transformer	1073	1.7170	0.0284
core factorial	Mamdani, 3 rules, 1 layer(s), window 10	1049	1.5904	0.0476
core factorial	FANTF+Mamdani, 5 rules, 1 layer(s), window 5	1042	1.6082	0.0497
core factorial	Mamdani, 7 rules, 1 layer(s), window 10	1113	1.6084	0.0403
core factorial	FANTF+Mamdani, 3 rules, 1 layer(s), window 10	1050	1.6120	0.0554
core factorial	FANTF+Mamdani, 7 rules, 1 layer(s), window 20	1194	1.6211	0.0197
core factorial	FANTF+Mamdani, 7 rules, 1 layer(s), window 5	1074	1.6212	0.0468
core factorial	Mamdani, 3 rules, 1 layer(s), window 5	1009	1.6225	0.0542
core factorial	Mamdani, 7 rules, 1 layer(s), window 5	1073	1.6238	0.0525
core factorial	Mamdani, 5 rules, 1 layer(s), window 5	1041	1.6249	0.0503
core factorial	FANTF+Mamdani, 7 rules, 2 layer(s), window 5	2067	1.6254	0.1073
core factorial	Mamdani, 5 rules, 1 layer(s), window 10	1081	1.6299	0.0442
core factorial	FANTF+Mamdani, 7 rules, 1 layer(s), window 10	1114	1.6303	0.0502
core factorial	FANTF+Mamdani, 5 rules, 1 layer(s), window 10	1082	1.6327	0.0404
core factorial	FANTF+Mamdani, 3 rules, 1 layer(s), window 5	1010	1.6365	0.0664
core factorial	FANTF+Mamdani, 5 rules, 2 layer(s), window 5	2003	1.6401	0.1037
core factorial	Mamdani, 3 rules, 2 layer(s), window 5	1937	1.6471	0.0722
core factorial	FANTF+Mamdani, 3 rules, 2 layer(s), window 5	1939	1.6521	0.0794
core factorial	FANTF+Mamdani, 3 rules, 2 layer(s), window 20	2059	1.6690	0.0380
core factorial	FANTF+Mamdani, 3 rules, 2 layer(s), window 10	1979	1.6705	0.0879
core factorial	Mamdani, 7 rules, 1 layer(s), window 20	1193	1.6705	0.0721
core factorial	Mamdani, 5 rules, 2 layer(s), window 5	2001	1.6736	0.0942
core factorial	FANTF+Mamdani, 3 rules, 1 layer(s), window 20	1130	1.6754	0.0594
core factorial	FANTF+Mamdani, 7 rules, 2 layer(s), window 10	2107	1.6796	0.0606
core factorial	FANTF+Mamdani, 5 rules, 1 layer(s), window 20	1162	1.6893	0.0510
core factorial	Mamdani, 5 rules, 1 layer(s), window 20	1161	1.6893	0.0696
core factorial	Mamdani, 7 rules, 2 layer(s), window 5	2065	1.6991	0.0996
core factorial	Mamdani, 3 rules, 2 layer(s), window 20	2057	1.7282	0.0692
core factorial	FANTF+Mamdani, 5 rules, 2 layer(s), window 20	2123	1.7300	0.0312
core factorial	Mamdani, 3 rules, 2 layer(s), window 10	1977	1.7369	0.0580
core factorial	Mamdani, 3 rules, 1 layer(s), window 20	1129	1.7382	0.0550
core factorial	Mamdani, 7 rules, 2 layer(s), window 20	2185	1.7458	0.0904
core factorial	Mamdani, 5 rules, 2 layer(s), window 10	2041	1.7476	0.0448
core factorial	FANTF+Mamdani, 7 rules, 2 layer(s), window 20	2187	1.7562	0.0310
core factorial	Mamdani, 7 rules, 2 layer(s), window 10	2105	1.7688	0.0371
core factorial	FANTF+Mamdani, 5 rules, 2 layer(s), window 10	2043	1.7761	0.0667
core factorial	Mamdani, 5 rules, 2 layer(s), window 20	2121	1.8109	0.0334
curve pairs	parabolic -> gaussian	1009	1.6099	0.0470
curve pairs	trapezoidal -> triangular	1009	1.6120	0.0427
curve pairs	triangular -> gaussian	1009	1.6123	0.0456
curve pairs	trapezoidal -> parabolic	1009	1.6131	0.0449
curve pairs	gaussian -> parabolic	1009	1.6137	0.0451
curve pairs	gaussian -> triangular	1009	1.6146	0.0460
curve pairs	gaussian -> trapezoidal	1009	1.6160	0.0467
curve pairs	parabolic -> trapezoidal	1009	1.6165	0.0493
curve pairs	gaussian -> gaussian	1009	1.6166	0.0473
curve pairs	trapezoidal -> gaussian	1009	1.6175	0.0449
curve pairs	triangular -> trapezoidal	1009	1.6188	0.0530
curve pairs	trapezoidal -> trapezoidal	1009	1.6191	0.0489

Family	Configuration	Params	RMSE	SD
curve pairs	parabolic -> triangular	1009	1.6225	0.0542
curve pairs	parabolic -> parabolic	1009	1.6247	0.0608
curve pairs	triangular -> parabolic	1009	1.6262	0.0496
curve pairs	triangular -> triangular	1009	1.6268	0.0498
encoder depth	1 encoder layer(s)	1009	1.6225	0.0542
encoder depth	2 encoder layer(s)	1937	1.6471	0.0722
encoder depth	3 encoder layer(s)	2865	1.7656	0.0798
fantf delta	delta 0.1	1010	1.6141	0.0530
fantf delta	delta 0.25	1010	1.6234	0.0567
fantf delta	delta 0.5	1010	1.6365	0.0664
fantf delta	delta 1	1010	1.6386	0.0691
fuzzy placement	after_feedforward	1009	1.6225	0.0542
fuzzy placement	after_attention	1009	1.6296	0.0566
fuzzy placement	both	1065	1.6419	0.0583
sequence length	window 10	1049	1.5904	0.0476
sequence length	window 5	1009	1.6225	0.0542
sequence length	window 20	1129	1.7382	0.0550
sequence length	window 30	1209	1.7984	0.0722

Appendix B. Reproducibility and references

Primary commands

Purpose	Command
Full study	/Users/sam/anaconda3/bin/python3 run_comprehensive_study.py
Unit tests	/Users/sam/anaconda3/bin/python3 -m unittest discover -s tests -v
Build report	.venv/bin/python build_comprehensive_study_report.py

Evidence artifacts

Artifact	Path
Screening summary	outputs/comprehensive_study/screening_summary.csv
Raw screening folds	outputs/comprehensive_study/screening_runs.csv
Confirmation summary	outputs/comprehensive_study/confirmation_summary.csv
Raw confirmation folds	outputs/comprehensive_study/confirmation_runs.csv
Held-out seed metrics	outputs/comprehensive_study/heldout_summary.csv
Machine-readable synthesis	outputs/comprehensive_study/summary.json
Implementation	src/ierspop_transformer/model.py

References

#	Reference
1	Vaswani et al. Attention Is All You Need. arXiv:1706.03762.
2	Chakraborty and Heintz. Enhancing Time Series Forecasting with Fuzzy Attention-Integrated Transformers. arXiv:2504.00070; code: https://github.com/sanjaylopa22/FANTF
3	Nie et al. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. arXiv:2211.14730.
4	Liu et al. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. arXiv:2310.06625.
5	Mamdani and Assilian. An Experiment in Linguistic Synthesis with a Fuzzy Logic Controller. DOI: 10.1016/S0020-7373(75)80002-2.
6	Harvard NLP. The Annotated Transformer. https://github.com/harvardnlp/annotated-transformer

Reproduction note: every summary table comes from the saved CSV and JSON result files in this repository. The original dataset is not stored in GitHub, so it must be provided separately to rerun training.